

DEEP REINFORCEMENT LEARNING

365.382 (SS26)

Model-based RL: Planning with a Model



Sohvi Luukkonen

LIT AI LAB

Institute for Machine Learning

Recap and Motivation

Offline

No data collection

Offline Imitation Learning

Behavior cloning

Offline RL

Only use offline data
CQL, IQL, DT

Online

Policy data collection

Online Imitation Learning

DAgger

Requires expert data.
Doesn't need rewards.

Off-policy RL

Reuse data from
other policies

Replay buffer

DQN, DDPG,
SAC

Mult. grad step

PPO, Importance
sampling

On-policy RL

Only data from
current policy

REINFORCE

Critic only, Actor-Critic, Actor only

Inspired from Chelsea Finn.

Model-Free RL: What We Have So Far

- The agent interacts with the environment
- It observes (s, a, r, s') tuples
- It updates its estimates — no explicit model of the world
- Sample efficiency is limited: each real interaction is expensive (time, hardware, safety)

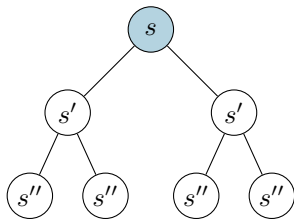
What if the agent could think ahead using an internal model of the world?

Motivation: Thinking Ahead

- Humans and animals **simulate** future outcomes before acting
- Chess players explore many candidate moves mentally
- Robots can plan trajectories without touching obstacles

Key question: can an RL agent do the same?

- Build or use a **model** of the environment
- Use it to **plan** without collecting real data
- Potentially massive gains in **sample efficiency**



Lecture Outline

Recap and Motivation

What Is a Model?

Planning vs. Learning

Dynamic Programming as Planning

Forward Search and Tree Search

Monte Carlo Tree Search

AlphaGo and AlphaZero

Trajectory Optimisation

Summary

What Is a Model?

What Is a Model?

A **model** of the environment is a learned or given function that predicts what happens when the agent acts.

Transition model:

$$s_{t+1} \sim p_{\phi}(s_{t+1} \mid s_t, a_t)$$

Reward model:

$$\hat{r} = r_{\phi}(s_t, a_t)$$

Together they define a **simulated MDP**: given any (s, a) , the agent can produce a prediction (s', r) without interacting with the real environment.

Types of Models

Axis	Option A	Option B
Origin	Known (given, e.g. game rules)	Learned from data
Output	Deterministic $s' = f(s, a)$	Stochastic $s' \sim p(\cdot s, a)$
State	Observation space	Latent space
Scope	Full model	Local / partial

- Classic planning (e.g. chess, Go): model is known exactly
- Model-based RL: model must be learned from data
- Latent models (e.g. MuZero): model operates in an abstract state space

What Can You Do with a Model?

A model enables several things that are impossible without one:

1. **Planning**: search over action sequences to find high-return trajectories
2. **Synthetic rollouts**: generate (s, a, r, s') without real interactions $\rightarrow$ use with any model-free algorithm (Dyna-style)
3. **Policy gradients through the model**: backpropagate through p_ϕ to get low-variance gradients (MBPO, SVG, Dreamer)
4. **Uncertainty-guided exploration**: explore states where the model is uncertain

This lecture focuses on (1): **planning** given a model.

Planning vs. Learning

Planning vs. Learning

Model-free learning

- Interact with the real environment
- Update value / policy from transitions (s, a, r, s')
- No explicit model
- Unbiased but **sample-hungry**

Planning

- Use a model to simulate transitions
- Search / optimise over imagined trajectories
- Model can be given or learned
- Fast once model is known, but **model errors propagate**

The two are **not mutually exclusive** — the most powerful agents do both.

Sample Efficiency: Model-Based vs Model-Free

- **Model-free methods** require millions of environment steps
- **Model-based methods** can achieve comparable performance with orders-of-magnitude fewer real interactions
- The gap is largest when:
 - ☐ Environment is expensive to simulate (robotics, chemistry)
 - ☐ Model is accurate (low-dimensional, smooth dynamics)
- **Caveat:** model errors can cause catastrophic policy updates

Dynamic Programming as Planning

Dynamic Programming as Full-MDP Planning

Dynamic Programming assumes the MDP is known:

$$p(s' \mid s, a), \quad r(s, a)$$

This allows us to compute Bellman backups without collecting new environment transitions.

DP is planning: use a model to improve values or policies.

But DP plans over the entire MDP:

- all states,
- all actions,
- all possible next states.

Two Classic DP Control Algorithms

Value Iteration: directly apply optimal Bellman backups.

$$V_{k+1}(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V_k(s') \right] \quad \forall s$$

Policy Iteration: alternate between policy evaluation and policy improvement.

Evaluation:
$$V^{\pi_k}(s) = \sum_a \pi_k(a | s) \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V^{\pi_k}(s') \right] \quad \forall s$$

Improvement:
$$\pi_{k+1}(s) = \arg \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V^{\pi_k}(s') \right] \quad \forall s$$

Why Not Just Use Dynamic Programming?

DP is conceptually clean, but it is rarely practical for large problems.

- **Explicit model required:** $p(s' | s, a)$ and $r(s, a)$
- **Full-state computation:** updates are performed for every state $s \in \mathcal{S}$.
- **Expensive backups:** each update sums over all possible next states:

$$\sum_{s'} p(s' | s, a) V(s')$$

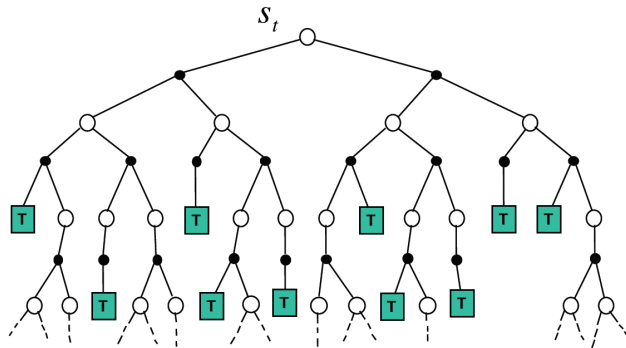
- **No generalization in tabular form:** similar states are treated as unrelated table entries.

Instead of solving the whole MDP, plan only from the current state.

Forward Search and Tree Search

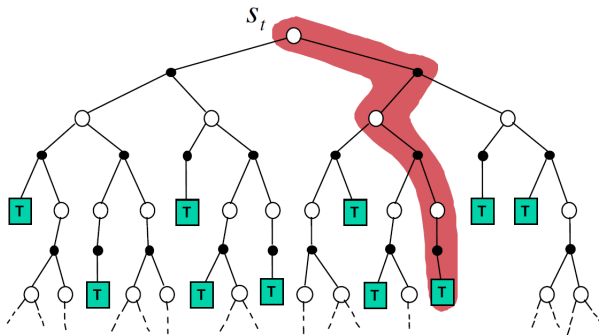
Forward Search from the Current State

- Forward search algorithms select the best action by **lookahead**
- They build a **search tree** with the current state s_t at the root
- Using a **model** of the MDP to look ahead
- No need to solve the whole MDP, just the sub-MDP starting **from now**



Simulation-Based Search

- **Sample-based** variant of forward search
- **Simulate** trajectories from s_t using the model $\hat{p}_\phi$ and policy π
- Apply any **model-free** method (MC, SARSA, Q-learning) to the simulated data



Simulation = using a model to generate imaginary experience.

Monte Carlo Simulation-Based Search

From the current state s_t , simulate K full episodes using model $\hat{p}_\phi$ and policy π :

$$\{s_t^k, a_t^k, r_{t+1}^k, s_{t+1}^k, \dots, s_T^k\}_{k=1}^K \sim \hat{p}_\phi, \pi$$

Monte-Carlo prediction — estimate state value from mean simulated return:

$$\hat{v}(s_t) = \frac{1}{K} \sum_{k=1}^K G_t^k \approx v^\pi(s_t)$$

Monte-Carlo control — estimate action value for each action a :

$$\hat{q}(s_t, a) = \frac{1}{K} \sum_{k=1}^K G_t^k \approx q^\pi(s_t, a), \quad A_t = \arg \max_a \hat{q}(s_t, a)$$

- **No global value function needed** — estimates are local to s_t
- Model is a **black box**: only needs to generate samples (r, s')

The Branching Problem

Full tree expansion becomes intractable very quickly.

At depth d with branching factor $b = |\mathcal{A}|$:

$$\text{nodes} = b^d$$

- Chess: $b \approx 35, d \approx 80 \Rightarrow 35^{80} \approx 10^{123}$ nodes
- Go: $b \approx 250, d \approx 150 \Rightarrow$ far beyond any computer

Two classical remedies:

- **Depth reduction:** replace deep rollouts with a value function $\hat{V}(s)$
- **Breadth reduction:** only expand promising branches (guided search)

Monte Carlo Tree Search addresses both.

Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS)

MCTS builds a partial lookahead tree using Monte Carlo simulations.

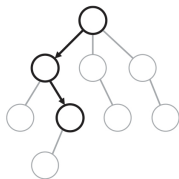
Four phases, repeated many times:

1. **Selection:** traverse from root to a leaf using a *tree policy* (UCT)
2. **Expansion:** add one or more child nodes to the selected leaf
3. **Simulation:** run a rollout from the new node to a terminal state using a fast *default policy* (e.g. random)
4. **Backpropagation:** update visit counts $N(s, a)$ and value estimates $Q(s, a)$ along the path back to the root

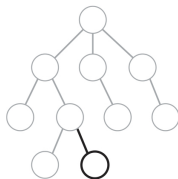
After the budget is exhausted, play the action with the **highest visit count** at the root.

Coulom, 2006; Kocsis & Szepesvári, 2006

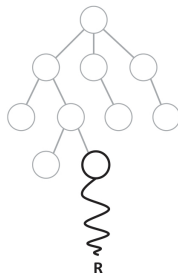
MCTS: Illustration



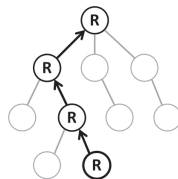
(a) Selection



(b) Expansion



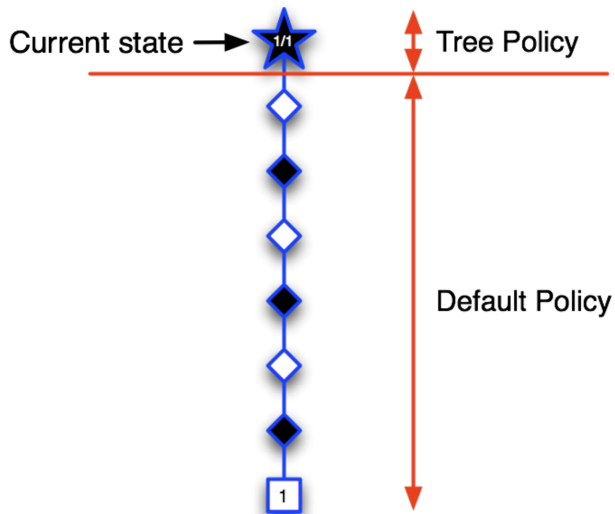
(c) Simulation



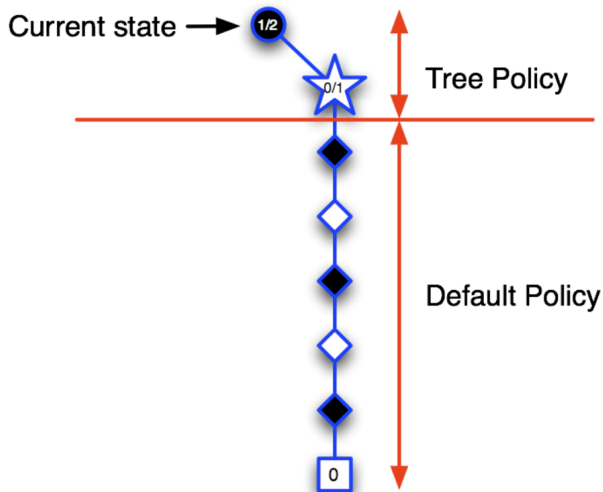
(d) Backpropagation

- The tree grows **asymmetrically**: promising subtrees are visited more
- Simulations provide **unbiased** value estimates (no value function needed)
- Over time, estimates converge to the true minimax value

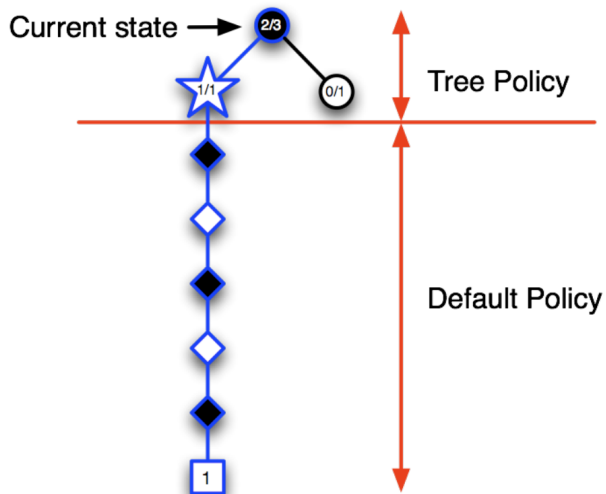
MCTS Example



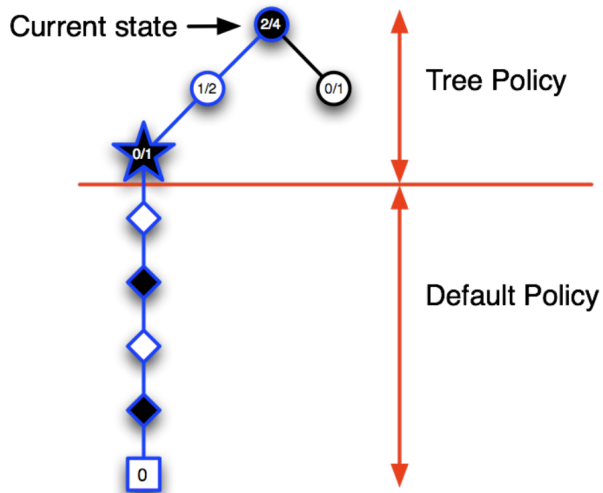
MCTS Example



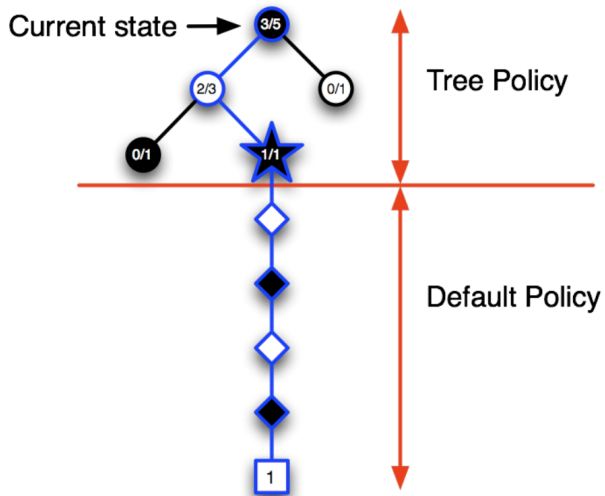
MCTS Example



MCTS Example



MCTS Example



UCT: The Tree Policy

How to decide which node to expand?

UCT (Upper Confidence Bound for Trees) applies UCB1 at each node:

$$a^* = \arg \max_a \left[Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right]$$

- $Q(s, a)$: mean return from simulations through (s, a)
- $N(s)$: total visits to node s ; $N(s, a)$: visits via action a
- c : exploration constant (hyperparameter)
- High $Q(s, a)$: **exploit** promising branches
- Low $N(s, a)$ relative to $N(s)$: **explore** under-visited branches

Kocsis & Szepesvári, 2006

MCTS Strengths

Theoretical guarantees:

- UCT converges to the minimax-optimal action as simulations $\rightarrow \infty$

Practical strengths:

- **Anytime**: more simulations can improve the decision
- **Selective**: computation focuses on promising branches
- **Black-box**: only needs a simulator, not explicit transition tables
- Can work without a learned policy or value function

MCTS Bottleneck: Leaf Evaluation

After selection and expansion, MCTS must estimate the value of a leaf state.

Classical solution: Monte Carlo rollout

$$\hat{V}(s_{\text{leaf}}) = \frac{1}{K} \sum_{k=1}^K G^k(s_{\text{leaf}})$$

Problem: in complex domains this estimate can be poor.

- Random rollout policy is often a weak evaluator
- Each rollout costs $\mathcal{O}(d)$ model queries
- Long rollouts have high variance
- Reliable estimates require many simulations

From Rollouts to Learned Value Functions

Key idea: replace expensive rollouts with a learned value function.

$$\frac{1}{K} \sum_{k=1}^K G^k(s_{\text{leaf}}) \longrightarrow V_{\theta}(s_{\text{leaf}})$$

The value function must be **trained from data**, e.g. game outcomes, Monte Carlo returns, TD targets, or self-play.

What V_{θ} provides:

- $\mathcal{O}(1)$ leaf evaluation
- generalisation across similar states
- compact long-horizon knowledge
- lower variance than random rollouts

What MCTS still provides:

- local decision-time computation
- comparison of candidate actions
- focused exploration of promising branches
- improved policy at the root

Neural MCTS combines **search** for local improvement
with **function approximation** for generalisation.

AlphaGo and AlphaZero

The Game of Go

- Go is 2500 years old, with simple rules but deep strategy
- The rules are known, but planning is still extremely difficult
- 19×19 board: about 10^{170} legal positions
- Large search tree: $b \approx 250$, $d \approx 150$
- Classical minimax-style search is infeasible

Why is Go so hard?

- Enormous branching factor
- Long-range positional strategy
- No simple hand-designed evaluation function



Go needs both [selective search](#) and [learned heuristics](#).

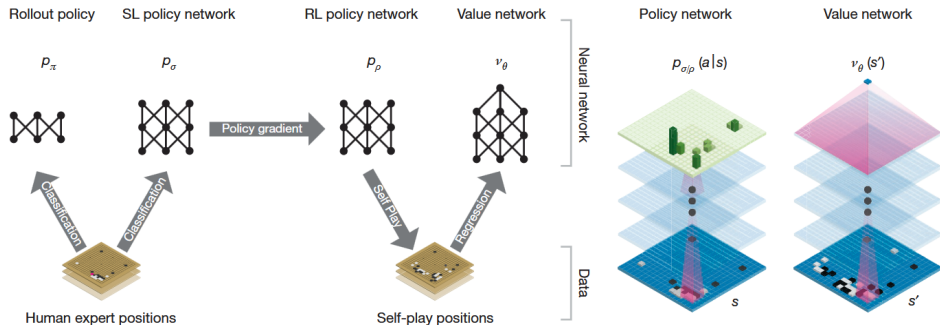
AlphaGo: MCTS + Deep Neural Networks

AlphaGo combined MCTS with learned policy and value functions:

- **Large policy network** $p_{\sigma}(a | s)$
 - ☐ first trained by supervised learning on expert human moves
 - ☐ then improved by self-play with policy gradients
 - ☐ used in MCTS as a prior over promising actions
- **Fast rollout policy** $p_{\pi}(a | s)$
 - ☐ much smaller and faster than the large policy network
 - ☐ used to simulate games from leaf states
- **Value network** $v_{\theta}(s)$
 - ☐ trained from self-play states to final game outcomes
 - ☐ estimates win probability from state s
 - ☐ used to evaluate leaf states

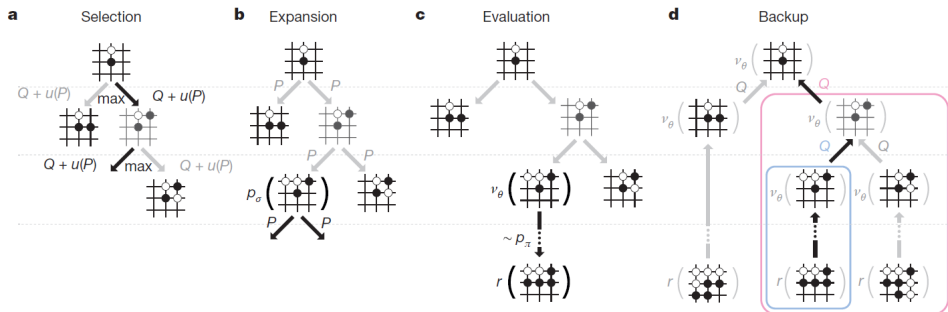
Silver et al., 2016

AlphaGo: Neural Network Training and Architectures



Silver et al., 2016

AlphaGo: MCTS with Neural Network Guidance



Modified UCT tree policy:

$$a^* = \arg \max_a \left[Q(s, a) + c \frac{p_\sigma(a | s)}{1 + N(s, a)} \right] \quad \text{with} \quad Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1} \mathbf{1}(s, a, i) V(s_L^i)$$

Silver et al., 2016

AlphaZero: Simpler Than AlphaGo

AlphaZero keeps the core idea of AlphaGo:

MCTS + policy/value networks + self-play

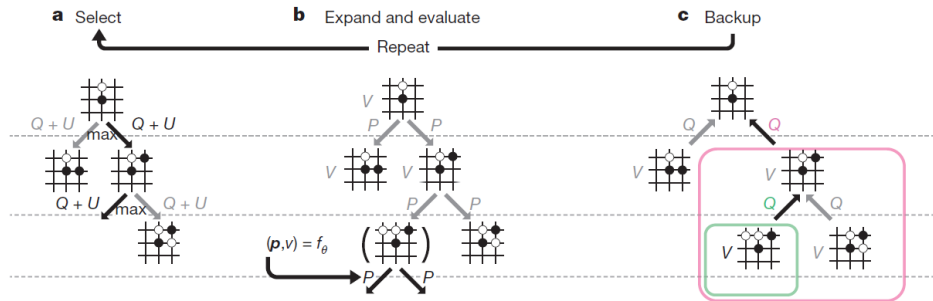
But removes several AlphaGo components:

- no human expert games
- no separate rollout policy
- no supervised pre-training
- one network predicts both policy and value:

$$f_{\theta}(s) = (\mathbf{p}, v)$$

AlphaZero learns only from self-play, using MCTS as an improvement step.

AlphaZero: MCTS with a Single Neural Network

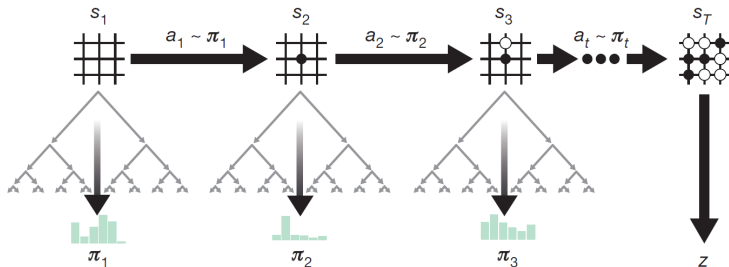


At each expanded node: $f_{\theta}(s) = (p, v)$

- p guides which actions to explore
- v evaluates the leaf state
- values are backed up through the tree

Search output: $\pi_{\text{MCTS}}(a \mid s) \propto N(s, a)^{1/\tau}$

AlphaZero: Self-Play Generates Targets

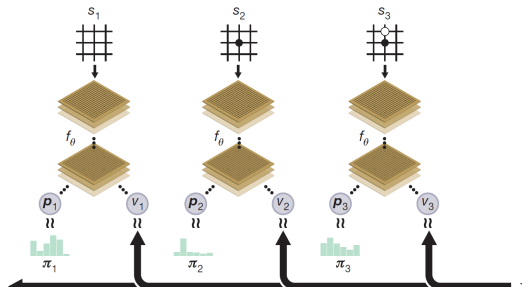


During self-play, MCTS is run at every state.

Each visited state gives one training example: $(s_t, \pi_{\text{MCTS}}(\cdot | s_t), z)$

- s_t : board position
- π_{MCTS} : improved search policy (MCTS visit distribution)
- z : final game outcome

AlphaZero: MCTS as a Teacher



The network is trained to predict both targets:

$$f_\theta(s_t) = (p_\theta(\cdot | s_t), v_\theta(s_t))$$

$$\mathcal{L}(\theta) = \underbrace{(v_\theta(s_t) - z)^2}_{\text{value loss (min. error)}} - \underbrace{\pi_{\text{MCTS}}^\top \log p_\theta}_{\text{policy loss (max. similarity)}} + \underbrace{c \|\theta\|^2}_{\text{regularization (weight decay)}}$$

JYU MCTS improves the policy; the network amortises the improved search.

AlphaZero: Key Takeaways

- Search improves the policy: $p_{\theta}(\cdot \mid s) \xrightarrow{\text{MCTS}} \pi_{\text{MCTS}}(\cdot \mid s)$
- Learning amortises search: the network is trained to predict the improved policy and final outcome.
- Self-play provides the curriculum: the opponent improves together with the agent.
- Two-player structure induces minimax-like reasoning: values are always interpreted from the current player's perspective.

Trajectory Optimisation

Trajectory Optimisation: Planning in Action Space

Tree search works well for discrete actions. In continuous control, branching over all actions is impossible.

Instead, plan by directly optimising an **action sequence**:

$$\mathbf{a}_{0:H-1}^* = \arg \max_{\mathbf{a}_{0:H-1}} \sum_{t=0}^{H-1} r(s_t, a_t), \quad s_{t+1} = f(s_t, a_t)$$

- f : queryable model of the dynamics
- H : planning horizon
- $\mathbf{a}_{0:H-1}$: candidate future actions
- objective: predicted return under the model

Trajectory optimisation searches over **action sequences**, not over tree nodes.

Model Predictive Control: Replanning Every Step

In **Model Predictive Control (MPC)**, the agent plans over a finite horizon, but only executes the first action.

At time t :

$$\mathbf{a}_{t:t+H-1}^* = \arg \max_{\mathbf{a}_{t:t+H-1}} \sum_{\tau=t}^{t+H-1} r(s_{\tau}, a_{\tau})$$

Execute only: $a_t = a_t^*$

Then observe the next real state s_{t+1} and re-plan.

- Corrects for model errors after each real transition
- Avoids committing to a long open-loop plan
- Naturally handles changing situations

Random Shooting

Simplest approach: sample many action sequences randomly, simulate them, and pick the best.

- 1: **for** $i = 1, \dots, N$ **do**
- 2: Sample candidate action sequence of length H : $\mathbf{a}_{0:H-1}^{(i)} \sim \mathcal{U}(\mathcal{A})^H$
- 3: Simulate the full plan with the model: $s_{t+1}^{(i)} = f(s_t^{(i)}, a_t^{(i)})$ for $t = 0, \dots, H - 1$
- 4: Compute its total return $J^{(i)} = \sum_t r(s_t^{(i)}, a_t^{(i)})$
- 5: **end for**
- 6: Execute 1st action of best sampled plan: $a = a_0^{(i^*)}$ where $i^* = \arg \max_i J^{(i)}$

- Simple and trivially parallelisable
- No gradient needed – model can be a black box
- **Limitation:** poor coverage in high-dimensional action spaces

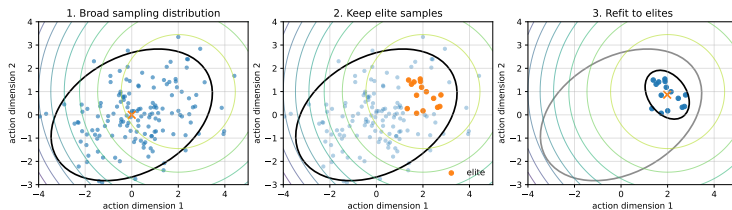
Cross-Entropy Method (CEM)

Idea: do not sample uniformly. Iteratively fit the sampling distribution to the best action sequences.

- 1: Initialise $\mu_0 = \mathbf{0}$, $\Sigma_0 = \sigma^2 \mathbf{I}$ over action sequences $\mathbf{a}_{0:H-1}$
- 2: **for** $k = 1, \dots, K$ **do**
- 3: **for** $i = 1, \dots, N$ **do**
- 4: Sample candidate action sequence of length H : $\mathbf{a}_{0:H-1}^{(i)} \sim \mathcal{N}(\mu_{k-1}, \Sigma_{k-1})$
- 5: Simulate the full plan with the model: $s_{t+1}^{(i)} = f(s_t^{(i)}, a_t^{(i)})$ **for** $t = 0, \dots, H-1$
- 6: Compute its total return: $J^{(i)} = \sum_t r(s_t^{(i)}, a_t^{(i)})$
- 7: **end for**
- 8: Select elite plans $\mathcal{E}$ with the highest returns
- 9: Refit distribution to elites: $\mu_k = \mathbb{E}_{\mathcal{E}}[\mathbf{a}]$, $\Sigma_k = \text{Cov}_{\mathcal{E}}[\mathbf{a}]$
- 10: **end for**
- 11: Execute first action of final mean plan: $a_{\text{current}} = \mu_{K,0}$

de Boer et al., 2005

CEM: Intuition



Strengths:

- Simple, robust, easy to implement
- Works with non-differentiable models
- Effective in low-to-medium action dimensions

Limitations:

- Gaussian assumption may miss multi-modal solutions
- Scales poorly to very high action dimensions

Comparison: Tree Search vs. Trajectory Optimisation

Property	Tree Search / MCTS	Shooting / CEM
Typical action space	Discrete	Continuous
Candidate object	Action tree	Action sequence
Model requirement	Queryable simulator	Queryable simulator
Search structure	Branching tree	Sampled trajectories
Planning horizon	Can be deep	Usually short, MPC-style
Guidance	UCT, policy prior, value net	Elite selection / refitting
Output	Root action distribution	First action of best sequence

Both are **online planners**: they spend computation at decision time using a model.

Summary

Summary: Planning with a Model

A model lets the agent evaluate possible futures before acting.

- **Dynamic Programming:** plan over the full MDP using Bellman backups
- **Forward search and MCTS:** plan locally from the current state using simulated futures
- **Neural MCTS:** use policy networks to guide search and value networks to evaluate leaves
- **Trajectory optimisation:** search directly over continuous action sequences
- **MPC:** execute only the first planned action, then re-plan

Planning trades **computation** for fewer real environment interactions.

What Changes When the Model Is Learned?

Today, we assumed a **queryable model**:

$$s_{t+1} = f(s_t, a_t) \quad \text{or} \quad s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)$$

In modern model-based RL, this model is often learned from data:

$$s_{t+1} \sim p_\phi(s_{t+1} \mid s_t, a_t)$$

This creates new problems:

- **Model error**: predictions become wrong
- **Compounding error**: small errors accumulate over rollouts
- **Distribution shift**: the planner may query unfamiliar states/actions
- **Model exploitation**: the policy may exploit model mistakes

Next lecture: learning models and controlling model error.